

数据结构与算法 A

来源：2026 年 9 月 9 日课程笔记

1. 本章要回答的四个问题

- 一个现实问题怎么变成写的程序？
- 数据之间的关系怎么描述，又怎么落到内存里？
- 有些运算和运算怎么实现怎么分开？
- 两个算法谁更快，凭什么下这个结论？

程序 = 数据结构 + 算法 (Wirth)。结构选错了，算法再巧也补不回来。

1.1. 问题描述：股市传言

若干股票经纪人之间互相传消息。谁能直接传给谁、要花多久，都是已知的。

问：从哪一个人开始散布消息，能让所有人最快都知道？（也就是最后知道消息的人今早得知）

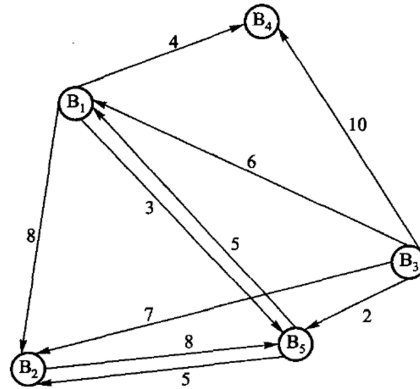


图 1.2 相邻矩阵：这就是算法的输入

	B1	B2	B3	B4	B5
B1	0	8	∞	4	3
B2	∞	0	∞	∞	8
B3	6	7	0	10	2
B4	∞	∞	∞	0	∞
B5	5	5	∞	∞	0

第 i 行第 j 列是边 $\langle B_i, B_j \rangle$ 的权；没有边记 ∞ ，对角线是 0

解法拆分为三步：

- Floyd: 先算出任意两人之间的最短传播时间；
- 每行取最大：消息要传遍所有人，就得等最慢的那个人收到
- 各行的最大值里取最小：这个起点就是答案
- 某一行还留着 ∞ ，有人永远收不到，这个起点不合格。

2. 代码实现

这里的逻辑很容易理解，那么，如何实现 Floyd 算法呢？我们可以用一个二维数组来表示每个人之间的传播时间，初始化时，如果两个人之间有直接的传递关系，就将对应的数组元素设为该传递时间，否则设为无穷大（表示无法直接传递）。然后，我们通过三重循环来更新这个二维数组，逐步计算出任意两人之间的最短传播时间。

2.1. 三重循环具体是什么？

Floyd 算法的核心思想是：

“如果允许经过中间点 k ，那么从 i 到 j 的距离是否会变得更短？”

设 $\text{dist}[i][j]$ 表示从节点 i 到节点 j 的当前最短距离。对于每个中间点 k ，尝试使用下面这条新路径：

$$i \rightarrow k \rightarrow j$$

如果这条路径更短，就更新 $\text{dist}[i][j]$ ：

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

对应的 C++ 三重循环如下：

```
for (int k = 1; k <= n; ++k) {
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            dist[i][j] = min(
                dist[i][j],
                dist[i][k] + dist[k][j]
            );
        }
    }
}
```

三个循环变量分别表示：

- k ：本轮允许经过的中间点；
- i ：路径的起点；
- j ：路径的终点。

2.1.1. 举个例子

假设当前有以下三条直接路径：

$1 \rightarrow 2$ 距离为 4

$2 \rightarrow 3$ 距离为 2

$1 \rightarrow 3$ 距离为 10

当 $k = 2$ 时，算法会尝试让节点 2 作为中间点。

从节点 1 经过节点 2 到达节点 3 的距离为：

$$\text{dist}[1][2] + \text{dist}[2][3] = 4 + 2 = 6$$

由于：

$$6 < 10$$

所以算法会将 $\text{dist}[1][3]$ 从 10 更新为 6。

```
"dist"[1][3] = min(10, 4 + 2);
// "dist"[1][3] 变为 6
```

2.1.2. 为什么 k 必须放在最外层？

完成第 k 轮循环后， $\text{dist}[i][j]$ 表示：

“从 i 到 j ，只允许使用编号为 1 到 k 的节点作为中间点时，能够得到的最短距离。”

算法会逐步扩大允许使用的中间点范围：

阶段	允许经过的中间点
初始化	不允许经过任何中间点
第 1 轮	允许经过节点 1
第 2 轮	允许经过节点 1, 2
...	...
第 n 轮	允许经过全部节点

因此，Floyd 算法的标准循环顺序是：

$$k \rightarrow i \rightarrow j$$

不能随意写成 $i \rightarrow j \rightarrow k$ ，因为只有先完成第 $k-1$ 轮，才能正确计算第 k 轮的最短路径。

2.1.3. 初始化距离数组

首先，将所有节点之间的距离初始化为无穷大，并将节点到自身的距离设为 0：

```
const long long INF = 4e18;

vector<vector<long long>> dist(
    n + 1,
    vector<long long>(n + 1, INF)
);

for (int i = 1; i <= n; ++i) {
    dist[i][i] = 0;
}
```

读入一条从 from 到 to 、传播时间为 time 的有向边：

```
int from, to;
long long time;

cin >> from >> to >> time;

dist[from][to] = min(dist[from][to], time);
```

这里使用 min ，是为了处理两个节点之间可能存在多条直接路径的情况。

如果题目中的关系是双向的，还需要添加：

```
dist[to][from] = min(dist[to][from], time);
```

2.1.4. 防止无穷大参与加法

如果 i 无法到达 k ，或者 k 无法到达 j ，那么就不存在路径：

$$i \rightarrow k \rightarrow j$$

因此可以直接跳过本次计算：

```
for (int k = 1; k <= n; ++k) {
    for (int i = 1; i <= n; ++i) {
        if (dist[i][k] == INF) {
            continue;
        }

        for (int j = 1; j <= n; ++j) {
            if (dist[k][j] == INF) {
                continue;
            }

            dist[i][j] = min(
                dist[i][j],
                dist[i][k] + dist[k][j]
            );
        }
    }
}
```

这样既能减少无效计算，也可以避免无穷大相加导致整数溢出。

2.1.5. 找到最佳消息起点

Floyd 算法结束后， $\text{dist}[i][j]$ 就表示从节点 i 到节点 j 的最短传播时间。

对于每个候选起点 i ，找出消息从 i 出发传到所有人所需的最长时间：

$$T_i = \max_{1 \leq j \leq n} \text{dist}[i][j]$$

然后，从所有能够到达其他节点的起点中，选择 T_i 最小的一个：

$$\text{答案} = \min_{1 \leq i \leq n} T_i$$

对应的 C++ 代码如下：

```
int answer = -1;
long long bestTime = INF;

for (int i = 1; i <= n; ++i) {
    long long lastArrival = 0;
    bool reachable = true;

    for (int j = 1; j <= n; ++j) {
        if (dist[i][j] == INF) {
            reachable = false;
            break;
        }

        lastArrival = max(lastArrival, dist[i][j]);
    }

    if (reachable && lastArrival < bestTime) {
        bestTime = lastArrival;
        answer = i;
    }
}
```

如果 `answer == -1`，说明不存在一个能够把消息传给所有人的起点：

```
if (answer == -1) {
    cout << "disjoint\n";
} else {
    cout << answer << ' ' << bestTime << '\n';
}
```

2.1.6. 复杂度分析

Floyd 算法需要枚举中间点、起点和终点，因此时间复杂度为：

$$O(n^3)$$

距离数组需要保存任意两个节点之间的距离，因此空间复杂度为：

$$O(n^2)$$

Floyd 算法适合节点数量不太多，并且需要一次性求出任意两个节点之间最短路径的场景。